Lập trình mạng · Chương 5

Độ tin cậy: retry, request ID và checksum trên UDP

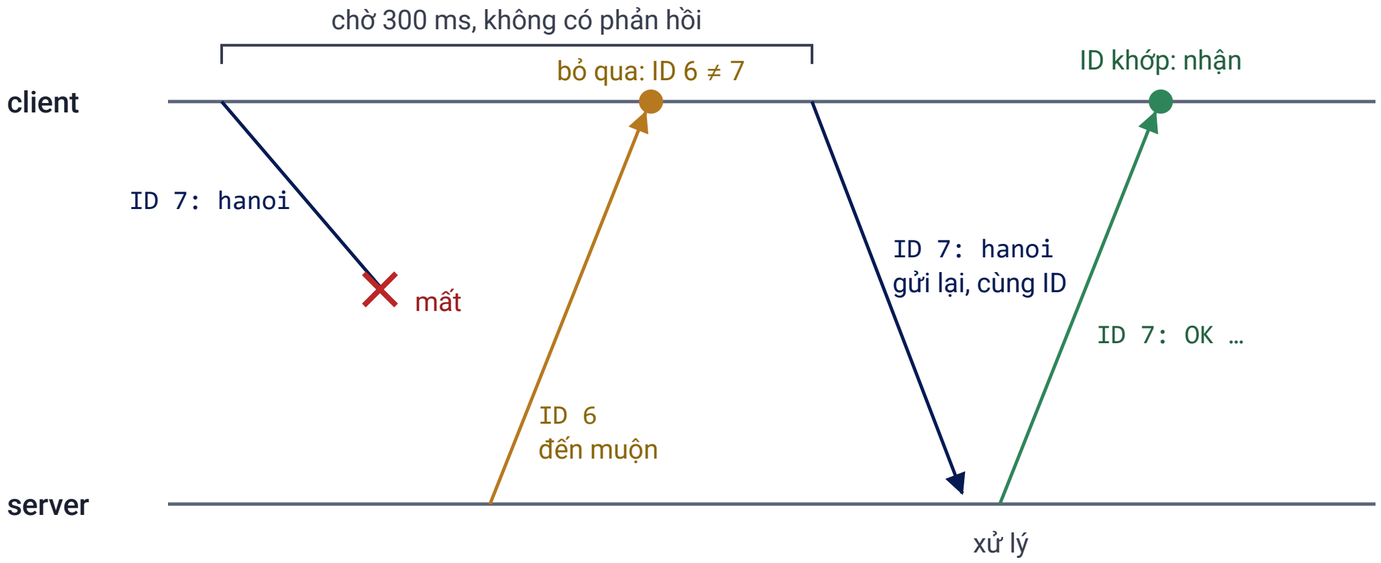
TS. Đặng Ngọc Hùng

Datagram có thể mất, lặp, đến muộn hoặc hỏng

1. Bản UDP ở Chương 3 giả định datagram **luôn đến**, đến đúng và chỉ một lần
2. Trên mạng thật, datagram có thể **mất**, **lặp**, **đến muộn** hoặc **hỏng**
3. QUIC xử lý ở tầng giao vận; giao thức tự định nghĩa trên UDP phải tự xử lý ở tầng ứng dụng

> **Câu hỏi của phần:** khi nào gửi lại, ghép phản hồi với yêu cầu thế nào, và nhận ra datagram hỏng bằng cách nào?

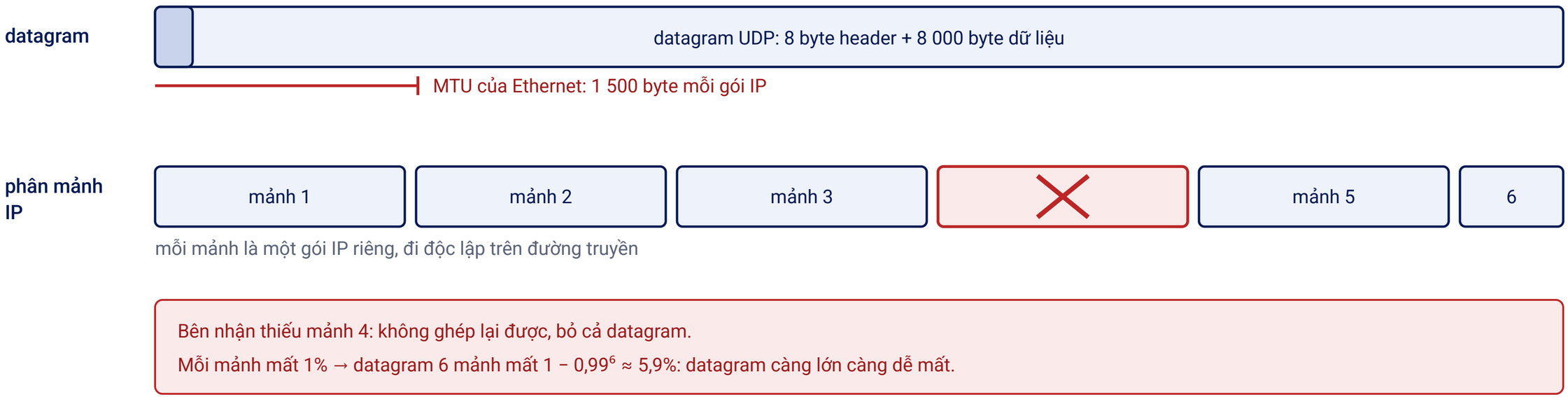
Retry có request ID: gửi lại cùng ID, chỉ nhận phản hồi khớp ID



Tối đa 3 lần gửi, mỗi lần chờ 300 ms;
sau 0,9 giây không có phản hồi
thì báo lỗi cho bên gọi.

Retry cùng **request ID** trong một deadline; phản hồi mang ID khác là phản hồi cũ đến muộn và bị bỏ qua.

Datagram lớn hơn MTU bị phân mảnh: mất một mảnh là mất cả datagram



Datagram lớn hơn **MTU** bị **phân mảnh IP**; mất một mảnh là mất cả datagram, nên giữ mỗi message gọn trong một gói IP.

CRC32: phát hiện datagram hỏng mà checksum của UDP bỏ sót

```java
b.putInt(id).put(body);
b.putInt((int) crc(b.array(), 4 + body.length));  // checksum của request ID và nội dung
if (b.getInt(length - 4) != (int) crc(data, length - 4)) {  // tính lại, so với CRC32 bên gửi ghi
    throw new CorruptException("CRC32 mismatch");
```

- Message: **request ID** 4 byte, nội dung UTF-8, **CRC32** 4 byte
- Checksum 16 bit của UDP chỉ bắt lỗi đơn giản, và có thể bị tắt ở IPv4
- CRC32 không khớp: bỏ datagram, xử lý như datagram mất

Client tra cứu trên mạng mất 30% datagram

Java

```
for (int send = 1; send <= MAX_SENDS; send++) {
    socket.send(new DatagramPacket(request, request.length, server));  // gửi lại: cùng request ID
        socket.setSoTimeout((int) left);
            reply = Message.decode(packet.getData(), packet.getLength());
        if (reply.id() != id) {
            System.out.println("       ignored late reply of id " + reply.id());  // phản hồi của yêu cầu cũ
        return reply.text();
```

Kết quả chạy

```
id 1   hanoi  OK Thủ đô của Việt Nam     sends 1
       dropped datagram: CRC32 mismatch
       dropped datagram: CRC32 mismatch
id 2   huế    (no reply after 3 sends)   sends 3
id 3   sapa   NOTFOUND                   sends 2
       ignored late reply of id 3
id 4   hanoi  OK Thủ đô của Việt Nam     sends 2
id 5   huế    OK Cố đô triều Nguyễn      sends 2
id 6   hanoi  OK Thủ đô của Việt Nam     sends 1
id 7   sapa   NOTFOUND                   sends 2
7 requests, 6 answered, 13 datagrams sent
```

Retry là at-least-once: chỉ an toàn với thao tác idempotent

```java
int times = processed.merge(request.id(), 1, Integer::sum);
```

Kết quả chạy

```
lossy UDP lookup on port 9199: loss 30%, late 15%, corrupt 10%
id 1: processed (time 1), replied
id 2: request lost
id 2: processed (time 1), reply corrupted
id 2: processed (time 2), reply corrupted
id 3: processed (time 1), reply sent 450 ms late
id 3: processed (time 2), replied
id 4: request lost
id 4: processed (time 1), replied
id 5: request lost
id 5: processed (time 1), replied
id 6: processed (time 1), replied
id 7: request lost
id 7: processed (time 1), replied
```

- Retry cho ngữ nghĩa **at-least-once**: server có thể xử lý một yêu cầu nhiều lần
- Tra cứu là **idempotent**; tăng bộ đếm thì server phải nhớ request ID đã xử lý

Câu hỏi 1

Client đang chờ phản hồi của yêu cầu ID 4 thì nhận được datagram hợp lệ mang ID 3. Client nên làm gì?

A. Nhận datagram đó làm kết quả của ID 4.

B. Báo lỗi cho bên gọi và dừng.

C. Bỏ qua datagram đó và chờ tiếp phản hồi mang ID 4.

D. Gửi lại yêu cầu ID 3.

Câu hỏi 1

Client đang chờ phản hồi của yêu cầu ID 4 thì nhận được datagram hợp lệ mang ID 3. Client nên làm gì?

A. Nhận datagram đó làm kết quả của ID 4.

B. Báo lỗi cho bên gọi và dừng.

C. Bỏ qua datagram đó và chờ tiếp phản hồi mang ID 4.

D. Gửi lại yêu cầu ID 3.

Datagram mang ID 3 là phản hồi đến muộn của yêu cầu trước; nhận nó làm kết quả của ID 4 là ghép sai phản hồi. Client bỏ qua và chờ tiếp trong thời gian còn lại.

Câu hỏi 2

Client retry khi hết thời gian chờ. Với thao tác nào, server cần nhớ request ID đã xử lý để không thực hiện hai lần?

A. Tra cứu giá trị của một khóa.

B. Đọc danh sách các khóa.

C. Ghi một giá trị cố định cho một khóa.

D. Tăng bộ đếm lượt xem của một khóa.

Câu hỏi 2

Client retry khi hết thời gian chờ. Với thao tác nào, server cần nhớ request ID đã xử lý để không thực hiện hai lần?

A. Tra cứu giá trị của một khóa.

B. Đọc danh sách các khóa.

C. Ghi một giá trị cố định cho một khóa.

D. Tăng bộ đếm lượt xem của một khóa.

Tăng bộ đếm không idempotent: xử lý hai lần thì bộ đếm tăng hai. Đọc và ghi một giá trị cố định đều idempotent, xử lý nhiều lần vẫn để lại cùng trạng thái.

Tổng kết

1. **Retry** cùng **request ID** trong một deadline; phản hồi khác ID bị bỏ qua
2. **Checksum** CRC32 cho mỗi message; message gọn trong một gói IP, tránh **phân mảnh IP** khi vượt **MTU**
3. Retry là **at-least-once**: chỉ an toàn với thao tác có **idempotency**

Từ khóa cần nhớ retry · request ID · idempotency · at-least-once · checksum · phân mảnh IP · MTU

Mở rộng

Chủ đề nâng cao

- Exponential backoff và jitter (phần sau)
- So sánh với cơ chế tin cậy của QUIC (Chương 3)

Bài tập mở rộng

Thêm lệnh INCR (không idempotent) vào dịch vụ UDP; server lưu request ID đã xử lý trong 30 giây để không tăng hai lần.

Tiếp theo

Độ tin cậy: backoff, circuit breaker và health check